

diffusion-cartogram: A Python package for diffusion deformations in 2 and 3 dimensions

Jonah Spector¹ and Albert-László Barabási^{1,2,3}

¹ Network Science Institute, Northeastern University, USA ² Department of Medicine, Brigham and Women's Hospital, Harvard Medical School, USA ³ Department of Network and Data Science, Central European University

DOI: [10.xxxxxx/draft](https://doi.org/10.xxxxxx/draft)

Software

- [Review](#)
- [Repository](#)
- [Archive](#)

Editor: [↗](#)

Submitted: 13 August 2026

Published: unpublished

License

Authors of papers retain copyright and release the work under a Creative Commons Attribution 4.0 International License ([CC BY 4.0](https://creativecommons.org/licenses/by/4.0/)).

Summary

diffusion-cartogram is a Python package for computing density-equalizing reference maps in both two and three dimensions. Given a spatial domain and a user-defined scalar density distribution, the package deforms the domain so that regions with higher density expand and regions with lower density contract, producing a smooth, orientation-preserving map that equalizes density across the domain.

In two dimensions, diffusion-cartogram implements a diffusion-based density-equalizing map for planar domains, enabling the construction of statistical cartograms from point clouds derived from shapefiles, GeoPandas GeoDataFrames, or plain CSV/XYZ files. In three dimensions, it implements the Volumetric Density-Equalizing Reference Map (VDERM) algorithm of Choi & Rycroft (2021), which extends this framework to fully volumetric domains in $\mathbb{R}^3$ using the reference map technique (Kamrin et al., 2012). Both modes are built on the same core architecture: a regular Cartesian grid, iterative backward Euler diffusion of the density field, and reference map tracking to recover the final deformation without per-iteration interpolation.

The package exposes both modes through a unified high-level API, includes built-in visualization utilities for animating and inspecting deformations, and supports export to XYZ, STL, and VTK formats for use in downstream tools such as ParaView. Example Jupyter notebooks cover a dependency-free 2D workflow (CSV inputs only), a world population cartogram, and 3D volumetric deformation examples.

Statement of Need

Density-equalizing maps are widely used in cartography, data visualization, and medical imaging, yet accessible Python implementations have been lacking for both planar and volumetric domains.

For two-dimensional cartograms, despite the foundational Gastner & Newman (2004) algorithm being over two decades old, no general-purpose Python implementation of diffusion-based contiguous cartograms existed prior to diffusion-cartogram. Existing tools are either implemented in other languages, limited to specific input formats, or not openly maintained. This gap is particularly notable given Python's dominance in data science and geospatial analysis workflows.

For three-dimensional volumetric deformations, the VDERM algorithm (Choi & Rycroft, 2021) was previously available only as a C++ prototype. No open Python implementation existed for data-driven 3D shape deformation via density equalization.

diffusion-cartogram fills both gaps in a single, pip-installable package. It is designed for researchers and practitioners in computational mathematics, cartography, geospatial data

41 science, medical imaging, and computational geometry who need density-driven spatial
42 deformations without requiring expertise in numerical PDE methods. A dependency-free mode
43 (using only NumPy and SciPy) ensures accessibility in restricted computational environments.

44 State of the Field

45 Density-equalizing maps originate from the work of Gastner & Newman (2004), who introduced
46 a diffusion-based algorithm for constructing contiguous cartograms—maps in which regions
47 are rescaled in proportion to a statistical quantity such as population. This method was later
48 extended to simply-connected open surfaces (Choi & Rycroft, 2018) and, most recently, to
49 genus-0 closed surfaces (Lyu et al., 2024).

50 The extension to volumetric domains was introduced by Choi & Rycroft (2021) via the reference
51 map technique (Kamrin et al., 2012), which tracks deformation on a fixed Eulerian grid and
52 requires interpolation only once, at the end of the iteration. This is a significant computational
53 advantage over naively extending the original Gastner & Newman (2004) approach to 3D. To
54 our knowledge, diffusion-cartogram is the first openly available Python implementation of
55 both the 2D diffusion-based cartogram and the 3D VDERM method.

56 Related open-source tools for spatial deformation include general-purpose mesh processing
57 libraries such as PyMeshLab (Muntoni & Cignoni, 2021) and PyVista (Sullivan & Kaszynski,
58 2019), as well as geometry processing packages like libigl (Jacobson et al., 2018). Geospatial
59 libraries such as GeoPandas (Jordahl et al., 2020) provide rich support for working with
60 geographic data but do not perform density-driven deformations. None of these tools provide
61 the density-equalizing deformation functionality of diffusion-cartogram.

62 Software Design

63 diffusion-cartogram is organized into parallel 2D and 3D modules (`core_2d` / `core` and
64 `visualization_2d` / `visualization`) that share a common design pattern across three stages:
65 grid initialization, density diffusion, and surface interpolation.

66 **Grid initialization.** A regular Cartesian grid is constructed over the bounding box of the input
67 geometry, with automatic sizing based on a target point count and optional padding. The
68 VDERMGrid class (2D and 3D variants) stores the reference map field and density field on this
69 grid. The density field is defined by a user-supplied callable $\rho(\mathbf{x})$, providing full flexibility in
70 specifying the deformation.

71 **Density diffusion and reference map update.** The core iteration alternates between solving the
72 diffusion equation for the density field using a backward Euler scheme with a pre-factored sparse
73 matrix, and updating the reference map using the resulting velocity field. No-flux boundary
74 conditions are enforced via ghost nodes; free boundary conditions are available by embedding
75 the domain in a uniform-density region. The iteration proceeds until a convergence criterion
76 on the maximum per-step displacement is met, or a maximum iteration count is reached.

77 **Surface interpolation.** After convergence, `interpolate_to_surface` maps input points to their
78 deformed positions using fast regular grid interpolation (`scipy.interpolate`). Performing
79 this interpolation once at the end—rather than at every iteration—is the key efficiency of the
80 reference map approach (Choi & Rycroft, 2021).

81 **Input and output.** The 2D module accepts point clouds from shapefiles, GeoPandas
82 GeoDataFrames, or plain CSV/XYZ files; a dependency-free mode (NumPy and SciPy only)
83 supports CSV inputs without requiring GeoPandas or Shapely. The 3D module supports point
84 cloud and mesh inputs via optional PyMeshLab integration. Both modules export to XYZ
85 format; the 3D module additionally supports STL and VTK export. Deformed meshes can be
86 reconstructed from point clouds via Poisson surface reconstruction.

Research Impact Statement

diffusion-cartogram makes density-equalizing map methods accessible to the broader scientific Python community for the first time in both two and three dimensions. The 2D module enables reproducible cartogram creation within standard geospatial Python workflows, while the 3D module opens the VDERM algorithm to researchers in medical imaging, computational geometry, and data visualization. Anticipated applications include population and demographic cartograms, data-driven adaptive remeshing of 3D domains, volumetric medical data visualization, and time-dependent shape morphing. Worked examples—including a world population cartogram and a 3D volumetric deformation tutorial—are provided as Jupyter notebooks to support immediate use in research pipelines.

AI Usage Disclosure

The software implementation of diffusion-cartogram was developed with the assistance of Claude (Anthropic), a large language model, used as a programming aid for code generation, debugging, and documentation. This paper was drafted with the assistance of Claude. All algorithmic decisions, design choices, validation, and scientific content are the responsibility of the authors. The original VDERM algorithm is the work of Choi & Rycroft (2021) and is unmodified in its mathematical substance by this implementation.

Acknowledgements

The authors thank Gary P. T. Choi and Chris H. Rycroft for the original development of the VDERM algorithm and for approving this Python implementation. Work by J.S. was supported in part by the National Institute of Health grant number T32 HL007427

References

- Choi, G. P. T., & Rycroft, C. H. (2018). Density-equalizing maps for simply connected open surfaces. *SIAM Journal on Imaging Sciences*, 11(2), 1134–1178. <https://doi.org/10.1137/17M1124796>
- Choi, G. P. T., & Rycroft, C. H. (2021). Volumetric density-equalizing reference map with applications. *Journal of Scientific Computing*, 86(3), 41. <https://doi.org/10.1007/s10915-021-01411-4>
- Gastner, M. T., & Newman, M. E. J. (2004). Diffusion-based method for producing density-equalizing maps. *Proceedings of the National Academy of Sciences*, 101(20), 7499–7504. <https://doi.org/10.1073/pnas.0400280101>
- Jacobson, A., Panozzo, D., & others. (2018). A flexible and easy-to-use mesh processing library. *ACM Transactions on Graphics*, 37(6), 1–4. <https://doi.org/10.1145/3272127.3275056>
- Jordahl, K., Van den Bossche, J., Fleischmann, M., & others. (2020). *GeoPandas: Python tools for geographic data* (Version v0.8.1). Zenodo. <https://doi.org/10.5281/zenodo.3946761>
- Kamrin, K., Rycroft, C. H., & Nave, J.-C. (2012). Reference map technique for finite-strain elasticity and fluid–solid interaction. *Journal of the Mechanics and Physics of Solids*, 60(11), 1952–1969. <https://doi.org/10.1016/j.jmps.2012.06.003>
- Lyu, Z., Lui, L. M., & Choi, G. P. T. (2024). Spherical density-equalizing maps for genus-0 closed surfaces. *SIAM Journal on Imaging Sciences*, 17(4), 2110–2141. <https://doi.org/10.1137/24M1633911>

- 128 Muntoni, A., & Cignoni, P. (2021). PyMeshLab. *Zenodo*. [https://doi.org/10.5281/zenodo.](https://doi.org/10.5281/zenodo.4438750)
129 [4438750](https://doi.org/10.5281/zenodo.4438750)
- 130 Sullivan, C. B., & Kaszynski, A. (2019). PyVista: 3D plotting and mesh analysis through a
131 streamlined interface for the visualization toolkit (VTK). *Journal of Open Source Software*,
132 4(37), 1450. <https://doi.org/10.21105/joss.01450>

DRAFT